

Consumo de servicios web con REST API

Modalidad: Práctica de laboratorio, individual

1. Propósito de la actividad

Que el estudiante consuma servicios web a través de una REST API, primero con **Postman** y después con **Python** (lenguaje multiparadigma), interpretando respuestas en formato **JSON** y comparándolas con **XML**, para reafirmar los temas de la unidad:

- APIs y análisis de JSON.
- REST API utilizando Postman.
- REST API utilizando un lenguaje multiparadigma.
- Servicios web con API REST, JSON o XML.

2. Materiales

- Equipo de cómputo con acceso a Internet.
- Postman (aplicación de escritorio o web).
- Python 3 y un IDE (VS Code, PyCharm u otro).
- Librería requests (instalar con `pip install requests`).
- APIs públicas que **no requieren registro ni llave**:
 - o JSONPlaceholder: `https://jsonplaceholder.typicode.com` (API de práctica con recursos `posts`, `users`, `comments`, etc.).
 - o Open-Meteo: `https://api.open-meteo.com/v1/forecast` (pronóstico del clima).

*Nota: JSONPlaceholder **simula** las operaciones de escritura. Responde como si el POST, PUT o DELETE hubieran funcionado, pero los datos no se guardan realmente. Es normal que un GET posterior no muestre el cambio.*

3. Parte A. Repaso conceptual (20 min)

Responde en tu cuaderno o en un documento, con tus propias palabras:

1. ¿Qué es una API y qué significa que una API sea REST?

La API es la regla que permite que dos códigos se comuniquen y una API REST significa que sigue una reglas para diseñar servicios web fáciles de usar

2. Describe la diferencia entre cliente y servidor en una petición HTTP.

Cuando el cliente empieza la comunicación hacia y el servidor procesa lo solicitado y responde

3. Completa la tabla de métodos HTTP:

Método	Operación CRUD	Para qué se usa	Código de éxito típico
GET	Leer	Consulta la información o recupera	200
POST	Crea	Crea los recursos	201
PUT	Actualiza	Reemplaza un recurso existente	200
PATCH	Actualiza	Modifica parcialmente un recurso en ciertos campos	200
Método	Operación CRUD	Para qué se usa	Código de éxito típico
DELETE	Elimina	Elimina un recurso	200 - 204

4. Explica qué significan los códigos de estado:

200: todo salió bien

201: todo salió bien y creo algo nuevo

204: la petición fue correcta pero no hay nada que mostrar

400: la petición tiene un error o los datos están mal

401: Necesitas identificarte

404: Lo que buscas no existe o no se encontró

500: El servidor tuvo un problema interno

5. Escribe un objeto JSON que represente a un dispositivo de red con: nombre, dirección IP, modelo, estado (activo o inactivo) y una lista de interfaces. Indica qué tipo de dato es cada valor.

```
{
  "hostname": "R1-Core",
  "ips": "192.168.1.1",
  "estado": "activo",
  "interfaces": [
    {
      "name": "GigabitEthernet0/0",
      "name": "GigabitEthernet0/1",
      "name": "Serial0/0/0"
    }
  ]
}
```

}

4. Parte B. Consumo de la API con Postman (60 min)

Crea una **colección** en Postman llamada `Unidad2_Act2_TuNombre` y dentro de ella una petición por cada ejercicio. Toma captura de pantalla de la petición y de la respuesta (cuerpo, código de estado y encabezados).

#	Petición	URL	Qué debes observar
B1	GET lista de publicaciones	<code>https://jsonplaceholder.typicode.com/posts</code>	Código 200 y que el cuerpo es una lista (arreglo) de objetos.
B2	GET un recurso	<code>https://jsonplaceholder.typicode.com/posts/1</code>	Campos <code>userId</code> , <code>id</code> , <code>title</code> , <code>body</code> .
B3	GET con parámetros	<code>https://jsonplaceholder.typicode.com/posts?userId=3</code>	Solo regresa publicaciones del usuario 3. Anota cuántas son.
B4	GET recurso anidado	<code>https://jsonplaceholder.typicode.com/posts/1/comments</code>	Comentarios de la publicación 1.
B5	POST crear	<code>https://jsonplaceholder.typicode.com/posts</code>	En <i>Body</i> → <i>raw</i> → <i>JSON</i> envía un objeto con <code>title</code> , <code>body</code> y <code>userId</code> . Código 201 y un <code>id</code> nuevo.
B6	PUT reemplazar	<code>https://jsonplaceholder.typicode.com/posts/1</code>	Envía un objeto completo. Compara con B2.
B7	PATCH modificar	<code>https://jsonplaceholder.typicode.com/posts/1</code>	Envía solo <code>{"title": "Nuevo título"}</code> .
B8	DELETE eliminar	<code>https://jsonplaceholder.typicode.com/posts/1</code>	Código 200 y cuerpo vacío <code>{}</code> .
B9	GET recurso inexistente	<code>https://jsonplaceholder.typicode.com/posts/9999</code>	Código 404. Explica qué significa.
B10	GET con parámetros de consulta	<code>https://api.open-meteo.com/v1/forecast?latitude=20.59&longitude=-</code>	Temperatura y viento actuales de Querétaro.

#	Petición	URL	Qué debes observar
		100.39¤t=temperature_2m,wind_speed_10m	Identifica las unidades en el campo current_units.

B1

Edgar Harim Monasterio González's Workspace

Search

Invite Upgrade

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Try Native Git

Unidad2_Act2_EdgarHarim

GET B1

OverviewGET Primer peticiUnidad2_Act2_EdGET B1No Environment

Unidad2_Act2_EdgarHarim

GET B1

Send

One gateway for every AI and API call. Skip the one-off integrations — route LLMs, MCP servers, and APIs through Fabric instead. Learn more about Fabric

DocsParamsAuthorizationHeaders6BodyScriptsSettingsCookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

BodyCookiesHeaders22Test Results200 OK281 ms7.76 KB

JSONPreviewVisualize

```
1 [
2   {
3     "userId": 1,
4     "id": 1,
5     "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
6     "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
7   },
8   {
9     "userId": 1,
10    "id": 2,
11    "title": "aut est esse"
12  }
13 ]
```

Add a JSON response schema test
Parameterize endpoint path
Set up a collection runner dataset
Describe what you need. Press @ for context, / for Skills.

B2

Edgar Harim Monasterio González's Workspace

Search

Invite Upgrade

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Try Native Git

Unidad2_Act2_EdgarHarim

GET B1

GET B2

OverviewGET Primer petiUnidad2_Act2GET B1GET B2No Environment

Unidad2_Act2_EdgarHarim

GET B1

GET B2

Send

One gateway for every AI and API call. Skip the one-off integrations — route LLMs, MCP servers, and APIs through Fabric instead. Learn more about Fabric

DocsParamsAuthorizationHeaders6BodyScriptsSettingsCookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

BodyCookiesHeaders22Test Results200 OK220 ms1.08 KB

JSONPreviewVisualize

```
1 {
2   "userId": 1,
3   "id": 1,
4   "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
5   "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
6 }
```

Fix the broken base URL
Add a connection-failure test
Create a second request for /posts/{id}
Describe what you need. Press @ for context, / for Skills.

B3

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Act2_EdgarHarim > B2

GET https://jsonplaceholder.typicode.com/posts?userId=3

Send

One gateway for every AI and API call. Skip the one-off integrations — route LLMs, MCP servers, and APIs through Fabric instead. Learn more about Fabric ↗

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
✓ userId	3			
Key	Value	Description		

Body Cookies Headers 22 Test Results 200 OK · 77 ms · 1.86 KB Save Response

JSON Preview Visualize

```
1 [
2   {
3     "userId": 3,
4     "id": 21,
5     "title": "asperiores ea ipsam voluptatibus modi minima quia sint",
6     "body": "repellat aliquid praesentium dolorem quo/nsed totam minus non itaque\n\nihil labore molestiae sunt dolor eveniet hic recusandae veniam\ntempora et tenetur expedita sunt"
7   },
8   {
9     "userId": 3,
10    "id": 22,
11    "title": "dolor sint qui a velit ex\ndicabo nita nam"
```

Set up environment base URL
Add negative auth/headers check
Duplicate B2 for a known-good host

Describe what you need. Press @ for context, / for Skills.
Auto

B4

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Act2_EdgarHarim > B4

GET https://jsonplaceholder.typicode.com/posts/1/comments

Send

One gateway for every AI and API call. Skip the one-off integrations — route LLMs, MCP servers, and APIs through Fabric instead. Learn more about Fabric ↗

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 22 Test Results 200 OK · 78 ms · 1.53 KB Save Response

JSON Preview Visualize

```
1 [
2   {
3     "postId": 1,
4     "id": 1,
5     "name": "id labore ex et quam laborum",
6     "email": "Eliseo@gardner.biz",
7     "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
8   },
9   {
10    "postId": 1,
11    "id": 2
```

Parameterize posts query for reuse
Create a B3-B2 parity test pack
Finalize B3 request so it's runnable

Describe what you need. Press @ for context, / for Skills.
Auto

B5

Edgar Harim Monasterio González's Workspace

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Act2_EdgarHarim > B5

POST https://jsonplaceholder.typicode.com/posts

Send

One gateway for every AI and API call. Skip the one-off integrations — route LLMs, MCP servers, and APIs through Fabric instead. Learn more about Fabric ↗

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Before request

After response

1. Use JavaScript to write tests, visualize response, and more.

Body Cookies Headers 23 Test Results 201 Created · 265 ms · 1.09 KB Save Response

JSON Preview Visualize

```
1 {
2   "title": "Prueba de red",
3   "body": "Contenido de ejemplo",
4   "userId": 1,
5   "id": 181
6 }
```

ENVIRONMENTS DOCUMENTS SPECS MOCKS DATASETS TOOLSETS FLOWS

main Terminal Console Add post-response tests

Globals Vault Tools

New Chat

has valid JSON body", function ()

Accept Decline Think Harder

Fix B5 body to send JSON

Add assertions for POST response

Save variables for title/body

Describe what you need. Press @ for context, / for Skills.

Auto

B6

Edgar Harim Monasterio González's Workspace

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Act2_EdgarHarim > B6

PUT https://jsonplaceholder.typicode.com/posts/1

Send

One gateway for every AI and API call. Skip the one-off integrations — route LLMs, MCP servers, and APIs through Fabric instead. Learn more about Fabric ↗

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Auth type

No Auth

No Auth

This request does not use any authorization.

Body Cookies Headers 21 Test Results 200 OK · 125 ms · 999 B Save Response

JSON Preview Visualize

```
1 {
2   "id": 1,
3   "title": "Nuevo título actualizado",
4   "body": "Nuevo contenido completo del recurso",
5   "userId": 1
6 }
```

ENVIRONMENTS DOCUMENTS SPECS MOCKS DATASETS TOOLSETS FLOWS

main Terminal Console Standing by for launch

Globals Vault Tools

New Chat

Fix B6 PUT request body

Add an assertion for PUT

Create reusable variables for reuse

Describe what you need. Press @ for context, / for Skills.

Auto

B7

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Act2_EdgarHarim > B7

PATCH https://jsonplaceholder.typicode.com/posts/1

Send

One gateway for every AI and API call. Skip the one-off integrations — route LLMs, MCP servers, and APIs through Fabric instead. Learn more about Fabric ↗

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 21 Test Results

200 OK · 117 ms · 1.05 KB

Save Response

JSON Preview Visualize

```
1 {
2   "userId": 1,
3   "id": 1,
4   "title": "Nuevo titulo",
5   "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
6 }
```

Restore B7 path variables

Add PATCH request body for B7

Apply shared env variables to B7

Describe what you need. Press @ for context, / for Skills.

Auto

B8

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Act2_EdgarHarim > B8

DELETE https://jsonplaceholder.typicode.com/posts/1

Send

One gateway for every AI and API call. Skip the one-off integrations — route LLMs, MCP servers, and APIs through Fabric instead. Learn more about Fabric ↗

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 21 Test Results

200 OK · 262 ms · 904 B

Save Response

JSON Preview Visualize

```
1 {}
```

Add negative-case delete test

Parameterize post id

Check request headers/auth consistency

Describe what you need. Press @ for context, / for Skills.

Auto

B9

The screenshot shows the Postman interface with a workspace named "Edgar Harim Monasterio González's Workspace". The left sidebar lists collections, with "Unidad2_Act2_EdgarHarim" expanded, showing a list of GET requests from B1 to B9. The main panel displays a GET request to "https://jsonplaceholder.typicode.com/posts/9999". The response is a "404 Not Found" error with a status of 259 ms and 904 B. The response body is empty. The right sidebar shows a "New Chat" panel with a message "Fix the 404 error" and a "Describe what you need. Press @ for context, / for Skills." prompt.

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Act2_EdgarHarim > B9

GET https://jsonplaceholder.typicode.com/posts/9999

One gateway for every AI and API call. Skip the one-off integrations — route LLMs, MCP servers, and APIs through Fabric instead. Learn more about Fabric ↗

Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 22 Test Results 404 Not Found 259 ms · 904 B Save Response

Fix the 404 error

JSON Preview Debug with AI

1 {}

New Chat

Fix the 404 error

Update the URL to a valid resource ID.

URL

https://jsonplaceholder.typicode.com/posts/9999

https://jsonplaceholder.typicode.com/posts/1

Accept Decline Think Harder

Add 200+404 assertions

Fix pathVariables mapping

Create a small runner dataset

Describe what you need. Press @ for context, / for Skills.

Auto

B10

The screenshot shows the Postman interface with the same workspace. The left sidebar lists collections, with "Unidad2_Act2_EdgarHarim" expanded, showing a list of GET requests from B1 to B10. The main panel displays a GET request to "https://api.open-meteo.com/v1/forecast?latitude=20.59&longitude=-100.39¤t=temperature_2m,wind,...". The response is a "200 OK" status with a status of 901 ms and 427 B. The response body is a JSON object containing forecast data. The right sidebar shows a "New Chat" panel with a message "Add assertions on forecast fields" and a "Describe what you need. Press @ for context, / for Skills." prompt.

Work locally with Git: Open your working directory in Postman to keep your API artifacts in sync with your codebase.

Unidad2_Act2_EdgarHarim > B10

GET https://api.open-meteo.com/v1/forecast?latitude=20.59&longitude=-100.39¤t=temperature_2m,wind,...

One gateway for every AI and API call. Skip the one-off integrations — route LLMs, MCP servers, and APIs through Fabric instead. Learn more about Fabric ↗

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
latitude	20.59		
longitude	-100.39		
current	temperature_2m,wind_speed_10m		
Key	Value	Description	

Body Cookies Headers 5 Test Results 200 OK 901 ms · 427 B Save Response

JSON Preview Visualize

```
1 {
2   "latitude": 20.56239,
3   "longitude": -100.43347,
4   "generationtime_ms": 0.07224802946777344,
5   "utc_offset_seconds": 0,
6   "timezone": "GMT",
7   "timezone_abbreviation": "GMT",
8   "elevation": 1021.0,
9   "current_units": {
10    "time": "iso8601",
11    "interval": "seconds",
12    "format": "2m"
13  }
```

New Chat

Add assertions on forecast fields

Parameterize lat/lon with env vars

Run B10 across a coords CSV

Describe what you need. Press @ for context, / for Skills.

Auto

Preguntas de la Parte B

1. ¿Qué diferencia observaste entre PUT (B6) y PATCH (B7) en lo que se envía?
En b6 se envía el objeto con todos los campos y en b7 solo se envía el campo a modificar
2. ¿Qué encabezado Content-Type regresa el servidor y qué indica?
El Content-Type: application/json; charset=utf-8
3. En B10, ¿qué parte de la URL es la ruta y cuál son los parámetros de consulta (query params)?
<https://api.open-meteo.com/v1/forecast> y los parametros son ?latitude=20.59&longitude=-100.39¤t=temperature_2m,wind_speed_10m

5. Parte C. Consumo de la API con Python

Crea una carpeta `actividad_rest` y un archivo `cliente_rest.py` por cada ejercicio

C1. Primer GET

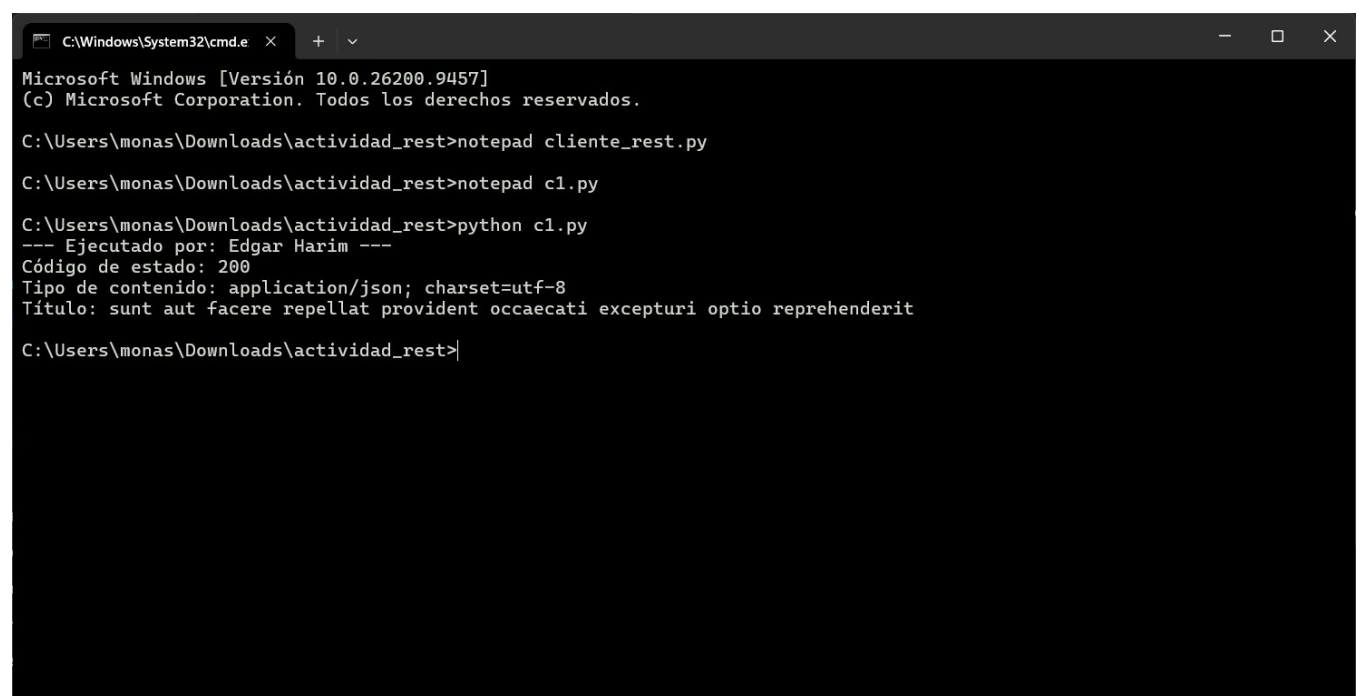
```
import requests

BASE="https://jsonplaceholder.typicode.com"

respuesta=requests.get(f"{BASE}/posts/1",timeout=10)

print("Código de estado:", respuesta.status_code)
print("Tipodecontenido:",respuesta.headers["Content-Type"])

publicacion = respuesta.json() #convierteelJSONenundiccionarioPython
print("Título:", publicacion["title"])
```



```
C:\Windows\System32\cmd.e  X  +  v

Microsoft Windows [Versión 10.0.26200.9457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\monas\Downloads\actividad_rest>notepad cliente_rest.py

C:\Users\monas\Downloads\actividad_rest>notepad c1.py

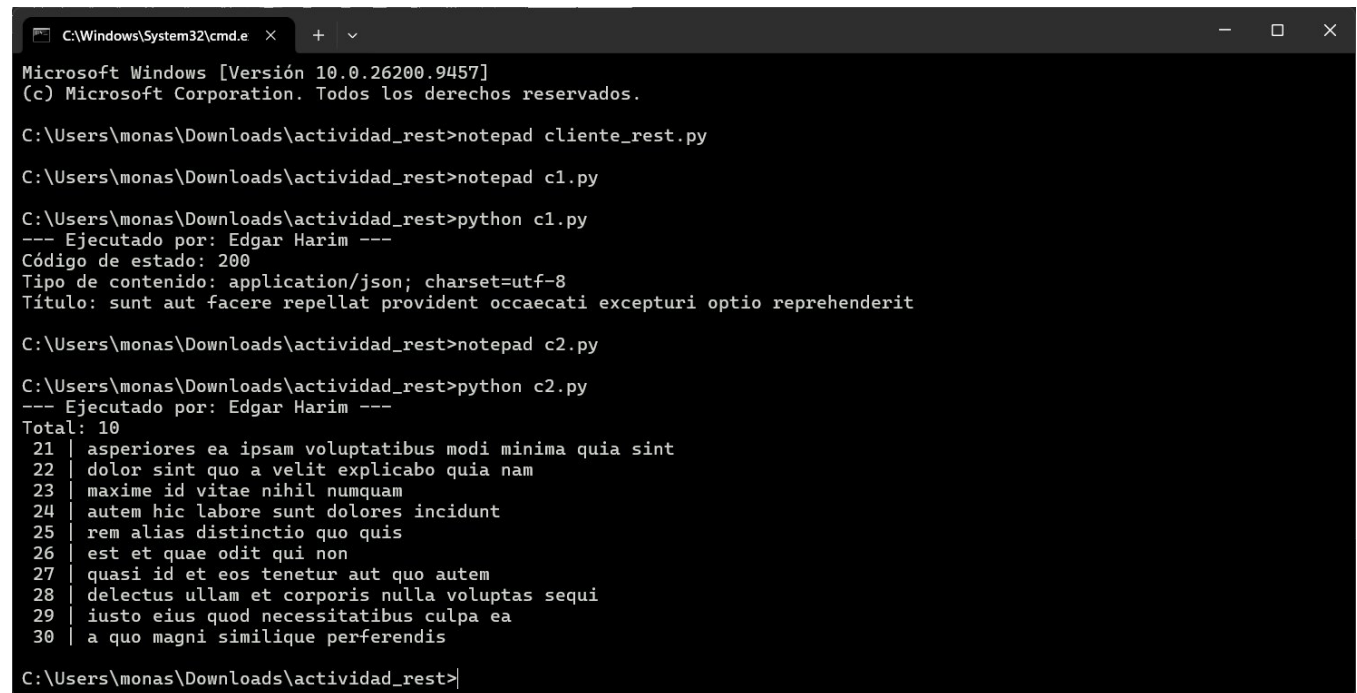
C:\Users\monas\Downloads\actividad_rest>python c1.py
--- Ejecutado por: Edgar Harim ---
Código de estado: 200
Tipo de contenido: application/json; charset=utf-8
Título: sunt aut facere repellat provident occaecati excepturi optio reprehenderit

C:\Users\monas\Downloads\actividad_rest>
```

C2. Recorrer una lista y usar parámetros

```
respuesta=requests.get(f"{BASE}/posts",params={"userId":3},timeout=10)
publicaciones = respuesta.json() # lista de diccionarios

print("Total:",len(publicaciones))
for p in publicaciones:
    print(f"{p['id']:>3}|{p['title']}")
```



The screenshot shows a Windows Command Prompt window with the following text:

```
C:\Windows\System32\cmd.e  X + v
Microsoft Windows [Versión 10.0.26200.9457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\monas\Downloads\actividad_rest>notepad cliente_rest.py

C:\Users\monas\Downloads\actividad_rest>notepad c1.py

C:\Users\monas\Downloads\actividad_rest>python c1.py
--- Ejecutado por: Edgar Harim ---
Código de estado: 200
Tipo de contenido: application/json; charset=utf-8
Título: sunt aut facere repellat provident occaecati excepturi optio reprehenderit

C:\Users\monas\Downloads\actividad_rest>notepad c2.py

C:\Users\monas\Downloads\actividad_rest>python c2.py
--- Ejecutado por: Edgar Harim ---
Total: 10
21 | asperiores ea ipsam voluptatibus modi minima quia sint
22 | dolor sint quo a velit explicabo quia nam
23 | maxime id vitae nihil numquam
24 | autem hic labore sunt dolores incidunt
25 | rem alias distinctio quo quis
26 | est et quae odit qui non
27 | quasi id et eos tenetur aut quo autem
28 | delectus ullam et corporis nulla voluptas sequi
29 | iusto eius quod necessitatibus culpa ea
30 | a quo magni similique perferendis

C:\Users\monas\Downloads\actividad_rest>
```

C3. POST, PUT y DELETE

```
nuevo={"title":"Pruebadered","body":"Contenidodeejemplo","userId":1}

r=requests.post(f"{BASE}/posts",json=nuevo,timeout=10)
print("POST:", r.status_code, r.json())

r=requests.put(f"{BASE}/posts/1",json=**nuevo,"id":1,timeout=10)
print("PUT:", r.status_code, r.json())

r=requests.delete(f"{BASE}/posts/1",timeout=10)
print("DELETE:", r.status_code)
```

```
C:\Windows\System32\cmd.e x + v
--- Ejecutado por: Edgar Harim ---
Código de estado: 200
Tipo de contenido: application/json; charset=utf-8
Titulo: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
C:\Users\monas\Downloads\actividad_rest>notepad c2.py
C:\Users\monas\Downloads\actividad_rest>python c2.py
--- Ejecutado por: Edgar Harim ---
Total: 10
21 | asperiores ea ipsam voluptatibus modi minima quia sint
22 | dolor sint quo a velit explicabo quia nam
23 | maxime id vitae nihil numquam
24 | autem hic labore sunt dolores incidunt
25 | rem alias distinctio quo quis
26 | est et quae odit qui non
27 | quasi id et eos tenetur aut quo autem
28 | delectus ullam et corporis nulla voluptas sequi
29 | iusto eius quod necessitatibus culpa ea
30 | a quo magni similique perferendis
C:\Users\monas\Downloads\actividad_rest>notepad c3.py
C:\Users\monas\Downloads\actividad_rest>python c3.py
--- Ejecutado por: Edgar Harim ---
POST: 201 {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 101}
PUT: 200 {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 1}
DELETE: 200
C:\Users\monas\Downloads\actividad_rest>
```

C4. Manejo de errores

Modifica C1 para que consulte /posts/9999 y maneje el error correctamente:

```
try:
    r=requests.get(f"{BASE}/posts/9999",timeout=10)
    r.raise_for_status()
except requests.exceptions.HTTPError as e:
    #lanzaexcept
    print(f"Error HTTP: {e}")
except requests.exceptions.RequestException as e:
```

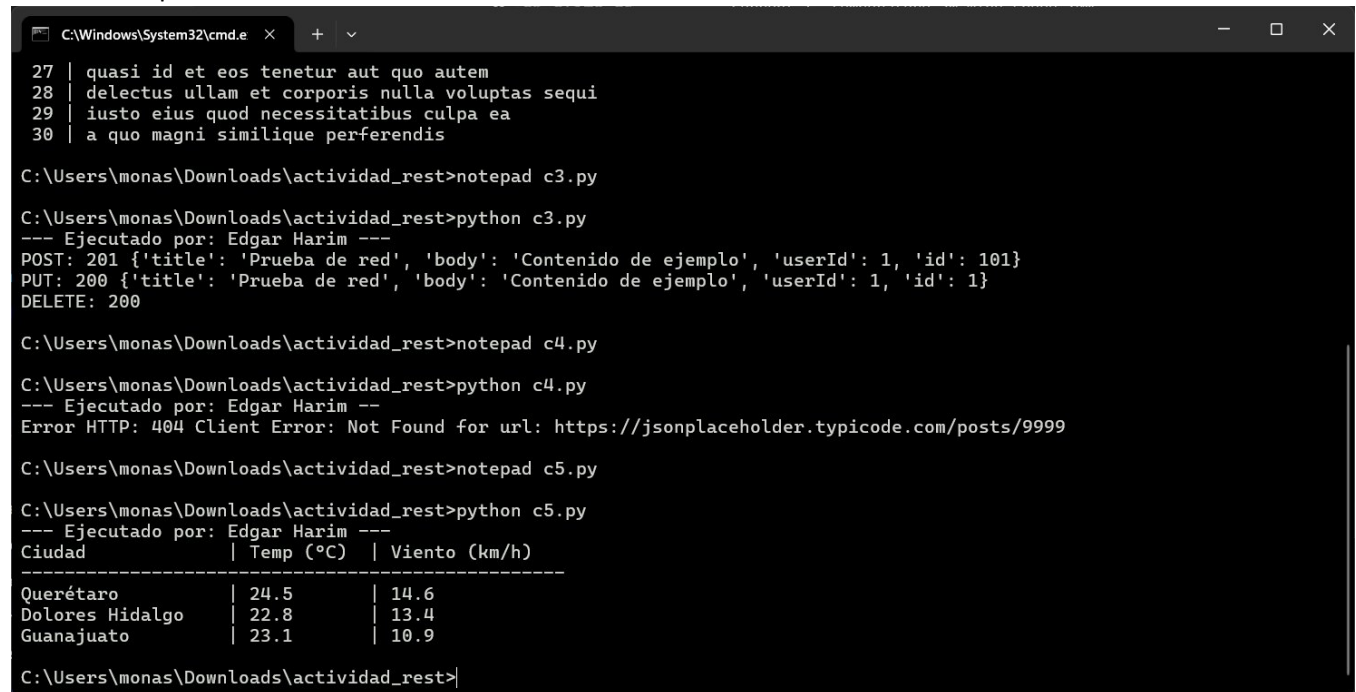
```
C:\Windows\System32\cmd.e x + v
C:\Users\monas\Downloads\actividad_rest>python c2.py
--- Ejecutado por: Edgar Harim ---
Total: 10
21 | asperiores ea ipsam voluptatibus modi minima quia sint
22 | dolor sint quo a velit explicabo quia nam
23 | maxime id vitae nihil numquam
24 | autem hic labore sunt dolores incidunt
25 | rem alias distinctio quo quis
26 | est et quae odit qui non
27 | quasi id et eos tenetur aut quo autem
28 | delectus ullam et corporis nulla voluptas sequi
29 | iusto eius quod necessitatibus culpa ea
30 | a quo magni similique perferendis
C:\Users\monas\Downloads\actividad_rest>notepad c3.py
C:\Users\monas\Downloads\actividad_rest>python c3.py
--- Ejecutado por: Edgar Harim ---
POST: 201 {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 101}
PUT: 200 {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 1}
DELETE: 200
C:\Users\monas\Downloads\actividad_rest>notepad c4.py
C:\Users\monas\Downloads\actividad_rest>python c4.py
--- Ejecutado por: Edgar Harim --
Error HTTP: 404 Client Error: Not Found for url: https://jsonplaceholder.typicode.com/posts/9999
C:\Users\monas\Downloads\actividad_rest>
```

C5. Consumir otra API (clima)

Escribe una función `obtener_clima(latitud, longitud)` que consulte Open-Meteo y regrese un diccionario con la temperatura y la velocidad del viento. Usa `params` en lugar de armar la URL a mano.

```
def obtener_clima(latitud, longitud):
    url="https://api.open-meteo.com/v1/forecast"
    parametros = {
        "latitude": latitud,
        "longitude": longitud,
        "current": "temperature_2m, wind_speed_10m",
    }
    r=requests.get(url, params=parametros, timeout=10)
    r.raise_for_status()
    return r.json()["current"]
```

Probarla con tres ciudades distintas (por ejemplo Querétaro: 20.59, -100.39) y mostrar los resultados en una tabla impresa en consola.



```
C:\Windows\System32\cmd.e x + v
27 | quasi id et eos tenetur aut quo autem
28 | delectus ullam et corporis nulla voluptas sequi
29 | iusto eius quod necessitatibus culpa ea
30 | a quo magni similique perferendis

C:\Users\monas\Downloads\actividad_rest>notepad c3.py

C:\Users\monas\Downloads\actividad_rest>python c3.py
--- Ejecutado por: Edgar Harim ---
POST: 201 {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 101}
PUT: 200 {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 1}
DELETE: 200

C:\Users\monas\Downloads\actividad_rest>notepad c4.py

C:\Users\monas\Downloads\actividad_rest>python c4.py
--- Ejecutado por: Edgar Harim ---
Error HTTP: 404 Client Error: Not Found for url: https://jsonplaceholder.typicode.com/posts/9999

C:\Users\monas\Downloads\actividad_rest>notepad c5.py

C:\Users\monas\Downloads\actividad_rest>python c5.py
--- Ejecutado por: Edgar Harim ---
Ciudad          | Temp (°C) | Viento (km/h)
-----
Querétaro      | 24.5      | 14.6
Dolores Hidalgo | 22.8      | 13.4
Guanajuato     | 23.1      | 10.9

C:\Users\monas\Downloads\actividad_rest>
```

6. Parte D. JSON frente a XML (30 min)

1. Toma la respuesta JSON de `/posts/1` y escribe a mano su equivalente en XML.
2. Con Python, convierte el diccionario a XML usando `xml.etree.ElementTree`:

```
import xml.etree.ElementTree as ET

raiz=ET.Element("publicacion")
for clave, valor in publicacion.items():
    ET.SubElement(raiz,clave).text=str(valor)

print(ET.tostring(raiz,encoding="unicode"))
```

3. Completa la tabla comparativa:

Criterio	JSON	XML
Legibilidad		

Tamaño del mensaje		
Tipos de datos		
Uso común en APIs REST		

7. Parte E. Mini proyecto integrador (60 min)

Desarrolla un script `reporte_api.py` que:

1. Consulte `/users` de JSONPlaceholder y obtenga el nombre, correo y ciudad (`address.city`) de cada usuario.
2. Para cada usuario, consulte cuántas publicaciones tiene (`/posts?userId=<id>`).
3. Consulte el clima actual de un punto geográfico de tu elección.
4. Maneje errores de conexión y de código de estado.
5. Guarde el resultado en un archivo `reporte.json` (usa `json.dump` con `indent=4` y `ensure_ascii=False`) y también imprima un resumen en pantalla.

Estructura sugerida del archivo de salida:

```
{
  "clima":{"temperature_2m":0.0,"wind_speed_10m":0.0}, "usuarios": [
    {"nombre":"...", "correo":"...", "ciudad":"...", "publicaciones":0}
  ]
}
```

8. Entregables (portafolio de evidencias)

1. Documento con las respuestas de la Parte A, las preguntas de la Parte B y la tabla de la Parte D.
2. Exportación de la colección de Postman (archivo `.json`) y capturas de pantalla de B1 a B10.
3. Archivos `.py` de las Partes C y E, con comentarios.
4. Archivo `reporte.json` generado por el mini proyecto.
5. Conclusión breve (5 a 8 líneas): ¿en qué casos de administración de redes serviría consumir una API REST?

9. Lista de verificación para la evaluación

Criterio	Cumple	No cumple
Identifica correctamente métodos HTTP y códigos de estado		
Ejecuta las 10 peticiones en Postman con evidencia		
Envía cuerpo JSON y parámetros de consulta correctamente		
Consume la API con Python usando <code>requests</code>		
Convierte respuestas JSON a estructuras de Python (diccionarios y listas)		

Criterio	Cumple	No cumple
Maneja errores y tiempos de espera (timeout, raise_for_status)		
Compara JSON y XML con un ejemplo funcional		
El mini proyecto genera el archivo reporte.json correcto		
Código ordenado, comentado y entregado a tiempo		